

Started on	Tuesday, 18 March 2025, 4:04 PM
State	Finished
Completed on	Wednesday, 19 March 2025, 10:49 AM
Time taken	18 hours 45 mins
Overdue	16 hours 45 mins
Grade	80.00 out of 100.00

Question **1**

Correct

Mark 20.00 out of 20.00

Write a python program to implement KMP (Knuth Morris Pratt).

For example:

Input	Result
ABABDABACDABABCABAB ABABCABAB	Found pattern at index 10

Answer: (penalty regime: 0 %)

Reset answer

```

1  def KMPSearch(pat, txt):
2      ##### Add your code here #####
3      #Start here
4      M = len(pat)
5      N = len(txt)
6      lps = [0]*M
7      j = 0
8      computeLPSArray(pat, M, lps)
9      i = 0
10     while (N - i) >= (M - j):
11         if pat[j] == txt[i]:
12             i += 1
13             j += 1
14         if j == M:
15             print ("Found pattern at index " + str(i-j))
16             j = lps[j-1]
17         elif i < N and pat[j] != txt[i]:
18             if j != 0:
19                 j = lps[j-1]
20             else:
21                 i += 1
22     #End here

```

	Input	Expected	Got	
✓	ABABDABACDABABCABAB ABABCABAB	Found pattern at index 10	Found pattern at index 10	✓
✓	SAVEETHAENGINEERING VEETHA	Found pattern at index 2	Found pattern at index 2	✓

Passed all tests! ✓

Correct

Marks for this submission: 20.00/20.00.

Question **2**

Correct

Mark 20.00 out of 20.00

Create a python program to find the Hamiltonian path using Depth First Search for traversing the graph .

For example:

Test	Result
hamiltonian.findCycle()	['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'A'] ['A', 'H', 'G', 'F', 'E', 'D', 'C', 'B', 'A']

Answer: (penalty regime: 0 %)

Reset answer

```

1 class Hamiltonian:
2     def __init__(self, start):
3         self.start = start
4         self.cycle = []
5         self.hasCycle = False
6
7     def findCycle(self):
8         self.cycle.append(self.start)
9         self.solve(self.start)
10
11    def solve(self, vertex):
12        ##### Add your code here #####
13        #Start here
14        if vertex == self.start and len(self.cycle) == N+1:
15            self.hasCycle = True
16            self.displayCycle()
17            return
18        for i in range(len(vertices)):
19            if adjacencyM[vertex][i] == 1 and visited[i] == 0:
20                nbr = i
21                visited[nbr] = 1
22                self.cycle.append(nbr)

```

	Test	Expected	Got	
✓	hamiltonian.findCycle()	['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'A'] ['A', 'H', 'G', 'F', 'E', 'D', 'C', 'B', 'A']	['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'A'] ['A', 'H', 'G', 'F', 'E', 'D', 'C', 'B', 'A']	✓

Passed all tests! ✓

Correct

Marks for this submission: 20.00/20.00.

Question 3

Correct

Mark 20.00 out of 20.00

Write a python program to find minimum steps to reach to specific cell in minimum moves by knight.

Answer: (penalty regime: 0 %)

Reset answer

```

1 class cell:
2
3     def __init__(self, x = 0, y = 0, dist = 0):
4         self.x = x
5         self.y = y
6         self.dist = dist
7
8     def isInside(x, y, N):
9         if (x >= 1 and x <= N and
10            y >= 1 and y <= N):
11             return True
12         return False
13     def minStepToReachTarget(knightpos,
14                               targetpos, N):
15         # add your code here
16         #Start here
17         dx = [2, 2, -2, -2, 1, 1, -1, -1]
18         dy = [1, -1, 1, -1, 2, -2, 2, -2]
19         queue = []
20         queue.append(cell(knightpos[0], knightpos[1], 0))
21         visited = [[False for i in range(N + 1)] for j in range(N + 1)]
22         visited[knightpos[0]][knightpos[1]] = True

```

	Input	Expected	Got	
✓	30	20	20	✓

Passed all tests! ✓

Correct

Marks for this submission: 20.00/20.00.

Question 4

Correct

Mark 20.00 out of 20.00

Write a Python program for Bad Character Heuristic of Boyer Moore String Matching Algorithm

For example:

Input	Result
ABAAAABCD ABC	Pattern occur at shift = 5

Answer: (penalty regime: 0 %)

Reset answer

```

1 NO_OF_CHARS = 256
2 def badCharHeuristic(string, size):
3     ##### Add your Code Here #####
4     #Start here
5     badChar = [-1]*NO_OF_CHARS
6     for i in range(size):
7         badChar[ord(string[i])] = i;
8     return badChar
9     #End here
10 def search(txt, pat):
11     m = len(pat)
12     n = len(txt)
13     badChar = badCharHeuristic(pat, m)
14     s = 0
15     while(s <= n-m):
16         j = m-1
17         while j>=0 and pat[j] == txt[s+j]:
18             j -= 1
19         if j<0:
20             print("Pattern occur at shift = {}".format(s))
21             s += (m-badChar[ord(txt[s+m])] if s+m<n else 1)
22         else:

```

	Input	Expected	Got	
✓	ABAAAABCD ABC	Pattern occur at shift = 5	Pattern occur at shift = 5	✓

Passed all tests! ✓

Correct

Marks for this submission: 20.00/20.00.

Question 5

Not answered

Mark 0.00 out of 20.00

Write a recursive python function to perform merge sort on the unsorted list of float values.

For example:

Test	Input	Result
mergesort(li)	5 3.2 1.5 1.6 1.7 8.9	[1.5, 1.6, 1.7, 3.2, 8.9]
mergesort(li)	6 3.1 2.3 6.5 4.5 7.8 9.2	[2.3, 3.1, 4.5, 6.5, 7.8, 9.2]

Answer: (penalty regime: 0 %)

1	
---	--